

分治

Meaninglessness

西安交通大学

July 11, 2026



目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

引例

n 个小球，其中的 $n-1$ 个等重，剩下 1 个次品重量稍轻，给一个无砝码天平，要求用最少的称量次数能够保证找出次品。

解法 1

随便拿出一个球放在天平左边，然后依次将其它球放在天平右边，当天平不平衡时，轻的那一边即为次品。

最坏情况下，需要比较 $n - 1$ 次。

解法 2

一次只能比较 **1** 个球实在是太慢了!

考虑将球分为三堆, 取出其中小球个数相等的两堆放在天平两侧。

- 如果天平平衡, 则次品在剩下的一堆中;
- 如果天平不平衡, 则次品在天平上较轻的那一堆中。

这样我们就排除了两堆球, 从更少的一堆球中找次品, 因此我们可以再对这一堆球使用上述策略, 直到这一堆球只剩一个, 那么剩下的这个球为次品。

每比较一次, 球的总数变为了原来的 $\frac{1}{3}$, 因此最终只需 $\lceil \log_3 n \rceil$ 次就可找到次品。

分治

- 分治 (**Divide and Conquer**), 即「分而治之」, 是把一个原本复杂而庞大的问题分成两个或更多个相同或相似的子问题, 直到最后子问题可以简单的直接求解, 最后再将子问题的解合并即得到原问题的答案。
- 其流程可以分为三步: 分解、治理、合并。
 - 1 分解原问题为结构相同的子问题, 并进行递归求解;
 - 2 当问题规模足够小, 可以直接解决时, 作为边界返回;
 - 3 合并并列的子问题的结果。

目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

二分查找

二分查找 (Binary Search), 是用来在一个有序数组中查找某一元素的算法。

例：给定长度为 n 的**严格升序**数组 $a_1 \sim a_n$ ，现给定一个数 x ，要求输出 x 在数组 a 中的下标，保证答案一定存在。

二分查找

我们能不能按照刚刚给小球分堆后比较的思想，来达到优化时间复杂度的目的？

记 $mid = \lfloor \frac{1+n}{2} \rfloor$ ，我们把整个数组划分为 $a_1 \sim a_{mid-1}$ ， a_{mid} 以及 $a_{mid+1} \sim a_n$ 三部分。我们有没有办法判断 x 在哪一部分当中呢？

二分查找

我们能不能按照刚刚给小球分堆后比较的思想，来达到优化时间复杂度的目的？

记 $mid = \lfloor \frac{1+n}{2} \rfloor$ ，我们把整个数组划分为 $a_1 \sim a_{mid-1}$ ， a_{mid} 以及 $a_{mid+1} \sim a_n$ 三部分。我们有没有办法判断 x 在哪一部分当中呢？

关键在于对数组 a 为升序的利用：

- 如果 $x < a_{mid}$ ，说明 $a_{mid+1} \sim a_n$ 都比 x 大，则 x 只能在 $a_1 \sim a_{mid-1}$ 中；
- 如果 $x = a_{mid}$ ，说明 x 在 a_{mid} 这一堆中，也就是 a_{mid} ；
- 如果 $x > a_{mid}$ ，说明 $a_1 \sim a_{mid-1}$ 都比 x 小，则 x 只能在 $a_{mid+1} \sim a_n$ 中。

这样我们就成功地减小了问题的规模，即从一个长数组变为了短数组，且短数组依然满足升序的性质。

这样一次查找的时间复杂度为 $\mathcal{O}(\log n)$ 。

二分查找

- 其流程可以分为三步：分解、治理、合并。
 - 1 分解原问题为结构相同的子问题，并进行递归求解；
 - 2 当问题规模足够小，可以直接解决时，作为边界返回；
 - 3 合并并列的子问题的结果。
- 在刚刚的二分查找中，分解、治理、合并分别代表了什么？
 - 1 分解：把长数组分解为两段短数组，且短数组与长数组的性质相同；
 - 2 治理：当数组长度为 1 时，那么 x 一定等于这个位置的值；
 - 3 合并：由于我们直接舍弃了另外一部分数组，只留下了一个子数组，因此没有合并过程。

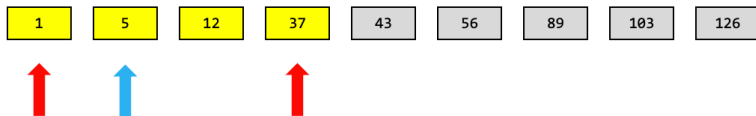
二分查找

$n = 9, x = 37, l = 1, r = 9, mid = 5$



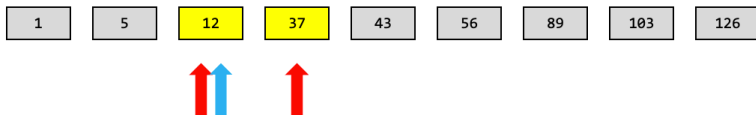
二分查找

$n = 9, x = 37, l = 1, r = 4, mid = 2$



二分查找

$n = 9, x = 37, l = 3, r = 4, mid = 3$



二分查找

$n = 9, x = 37, l = 4, r = 4, mid = 4$



二分查找

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 1e5 + 5;
4  int n, a[N];
5
6  int binary_search(int l, int r, int x) {
7      if(l == r)
8          return l;
9      int mid = (l + r) / 2;
10     if(x < a[mid])
11         return binary_search(l, mid - 1, x);
12     else if(x == a[mid])
13         return mid;
14     else
15         return binary_search(mid + 1, r, x);
16 }
17
18 int main() {
19     cin >> n;
20     for(int i = 1; i <= n; ++i)
21         cin >> a[i];
22     int x;
23     cin >> x;
24     cout << binary_search(1, n, x) << endl;
25     return 0;
26 }
```

二分查找

如果 x 不一定在数组 a 中怎么办? (此时要求返回 -1)

- 如果 $x < a_{mid}$, 说明 $a_{mid+1} \sim a_n$ 都比 x 大, 则 x 只能在 $a_1 \sim a_{mid-1}$ 中;
- 如果 $x = a_{mid}$, 说明 x 在 a_{mid} 这一堆中, 也就是 a_{mid} ;
- 如果 $x > a_{mid}$, 说明 $a_1 \sim a_{mid-1}$ 都比 x 小, 则 x 只能在 $a_{mid+1} \sim a_n$ 中。

二分查找

如果 x 不一定在数组 a 中怎么办? (此时要求返回 -1)

- 如果 $x < a_{mid}$, 说明 $a_{mid+1} \sim a_n$ 都比 x 大, 则 x 只能在 $a_1 \sim a_{mid-1}$ 中;
- 如果 $x = a_{mid}$, 说明 x 在 a_{mid} 这一堆中, 也就是 a_{mid} ;
- 如果 $x > a_{mid}$, 说明 $a_1 \sim a_{mid-1}$ 都比 x 小, 则 x 只能在 $a_{mid+1} \sim a_n$ 中。

在 $l = r$ 时, 先判断 x 是否等于 a_l , 如果相等返回 l , 不相等返回 -1 。

二分查找

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 1e5 + 5;
4 int n, a[N];
5
6 int binary_search(int l, int r, int x) {
7     if(l == r)
8         return a[l] == x ? l : -1;
9     int mid = (l + r) / 2;
10    if(x < a[mid])
11        return binary_search(l, mid - 1, x);
12    else if(x == a[mid])
13        return mid;
14    else
15        return binary_search(mid + 1, r, x);
16 }
17
18 int main() {
19     cin >> n;
20     for(int i = 1; i <= n; ++i)
21         cin >> a[i];
22     int x;
23     cin >> x;
24     cout << binary_search(1, n, x) << endl;
25     return 0;
26 }
```

二分查找

也可以写成循环的形式，相对于递归写法常数更小。

```
1 int binary_search(int x) {  
2     int l = 1, r = n;  
3     while(l < r) {  
4         int mid = (l + r) / 2;  
5         if(x < a[mid])  
6             r = mid - 1;  
7         else if(x == a[mid])  
8             return mid;  
9         else  
10            l = mid + 1;  
11     }  
12     return a[l] == x ? l : -1;  
13 }
```

二分查找

现在我们要求数列 a 中最小的满足大于等于 x 的下标位置，以及最小的满足大于 x 的下标位置，应该怎么办？

这里介绍 STL 中的两个函数：lower_bound 与 upper_bound

- lower_bound(first, last, value) 返回 $[first, last)$ 中第一个满足大于等于 x 的位置的地址/迭代器；
- upper_bound(first, last, value) 返回 $[first, last)$ 中第一个满足大于 x 的位置的地址/迭代器。

例题 1：查找

输入 n 个不超过 10^9 的单调不减的（就是后面的数字不小于前面的数字）非负整数 $a_1 \sim a_n$ ，然后进行 m 次询问。对于每次询问，给出一个整数 q ，要求输出这个数字在序列中第一次出现的编号，如果没有找到的话输出 -1 。

数据保证， $1 \leq n \leq 10^6$ ， $0 \leq a_i, q \leq 10^9$ ， $1 \leq m \leq 10^5$ 。

输入 #1

```
11 3
1 3 3 3 5 7 9 11 13 15 15
1 3 6
```

输出 #1

```
1 2 -1
```

例题 1：查找

可以采用 `lower_bound` 函数实现该题。记 t 为数组 a 中第一个大于等于 q 的下标。

- 如果 $t = n + 1$ ，说明所有数都小于 q ，即没有找到 q ，答案为 -1 ；
- 否则，满足 $a_1 \sim a_{t-1}$ 都比 q 小， $a_t \sim a_n$ 都**不小于** q ；
- 如果 $a_t \neq q$ ，则说明 $a_t \sim a_n$ 都大于 q ，答案为 -1 ；
- 否则答案为 t 。

时间复杂度为 $\mathcal{O}(n + m \log n)$ 。

例题 1：查找

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 1e6 + 5;
4  int n, m, a[N];
5
6  int main() {
7      cin.tie(nullptr) -> sync_with_stdio(false);
8      cin >> n >> m;
9      for(int i = 1; i <= n; ++i)
10         cin >> a[i];
11     while(m--) {
12         int q, t;
13         cin >> q;
14         t = lower_bound(a + 1, a + n + 1, q) - a;
15         if(t == n + 1 || a[t] != q)
16             cout << "-1_";
17         else
18             cout << t << '_';
19     }
20     cout.flush();
21     return 0;
22 }
```

目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

二分答案

二分查找是在**有序数组**中查找一个数。

二分答案则更进一步——把「**答案**」本身作为搜索目标，在答案的可能范围内进行二分。

核心思想

当问题的答案具有**单调性**时，可以将「求解」问题转化为「验证」问题。

使用二分答案需要满足以下三个条件：

- 1 答案在一个**固定区间**内；
- 2 判断一个值是否「可行」是**容易**的；
- 3 可行解具有**单调性**——如果 x 可行，则比 x 更大（或更小）的值也可行。

典型问法：「最大值最小」或「最小值最大」。

例题 2：数列分段 Section II

对于给定的一个长度为 n 的正整数数列 $a_1 \sim a_n$ ，现要将其分成 $m(m \leq n)$ 段，并要求每段连续，且每段和的最大值最小。

对于 100% 的数据， $1 \leq n \leq 10^5, m \leq n, a_i < 10^8$ ，答案不超过 10^9 。

输入 #1

```
5 3
4 2 4 5 1
```

输出 #1

```
6
```

例题 2：数列分段 Section II

定义 $f(x)$ 表示当每一段的和不超过 x 时，能否分成 m 段。

显然，当 x 越大，越有可能被划分为 m 段。接下来考虑找到 $f(x)$ 可行与不可行的分界点。

假设现在的区间为 $[l, r]$ ，判断 $mid = \lfloor \frac{l+r}{2} \rfloor$ 是否可行，这部分可以 $O(n)$ 模拟。

- 如果 $f(mid) = 1$ ，那么继续尝试区间 $[l, mid - 1]$ ；
- 否则，继续尝试区间 $[mid + 1, r]$ 。

时间复杂度为 $O(n \log n)$ 。

例题 2: 数列分段 Section II

```
1  const int N = 1e5 + 5;
2  int n, m, a[N];
3  bool check(int mid) {
4      int cnt = 1, tot = 0;
5      for(int i = 1; i <= n; ++i) {
6          if(tot + a[i] <= mid) tot += a[i];
7          else ++cnt, tot = a[i];
8      }
9      return cnt <= m;
10 }
11 int binary_answer(int l, int r) {
12     int ans = -1;
13     while(l <= r) {
14         int mid = (l + r) >> 1;
15         if(check(mid)) ans = mid, r = mid - 1;
16         else l = mid + 1;
17     }
18     return ans;
19 }
20 int main() {
21     cin >> n >> m;
22     int l = 0;
23     for(int i = 1; i <= n; ++i)
24         cin >> a[i], l = max(l, a[i]);
25     cout << binary_answer(l, 1e9) << endl;
26     return 0;
27 }
```

目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

归并排序

归并排序是基于分治实现的排序算法。

归并排序

归并排序是基于分治实现的排序算法。

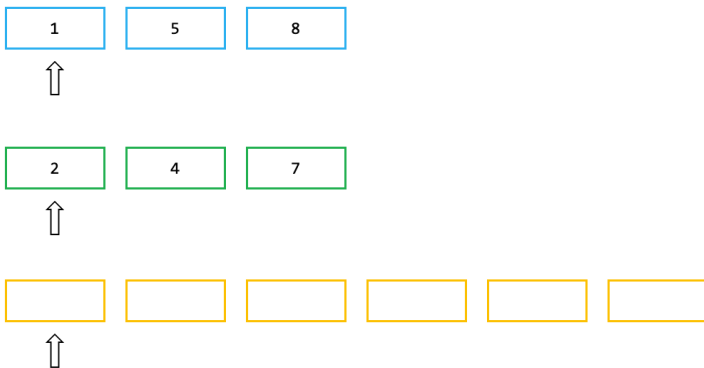
- 1 如果此时要排序的子数组长度为 1，则这段数组已经排好序，返回；
- 2 将整个数组先分解为前后两半数组，对其递归排序；
- 3 将前后两段排好序的数组合并为一个整体，并且也是排好序的。

归并排序

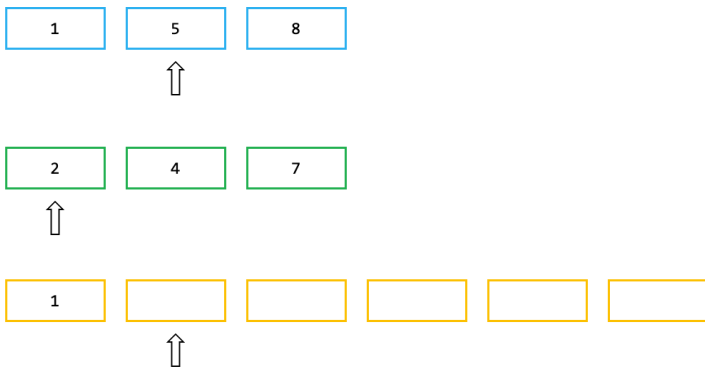
将前后两段排好序的数组合并为一个整体，并且也是排好序的。

例如：将 $\{1, 5, 8\}$ 和 $\{2, 4, 7\}$ 合并为 $\{1, 2, 4, 5, 7, 8\}$ 。

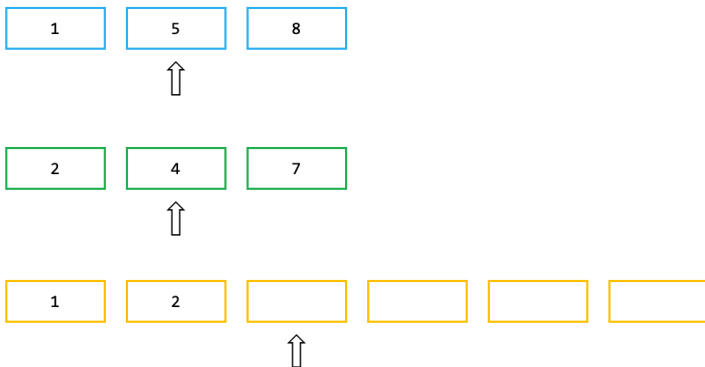
归并排序



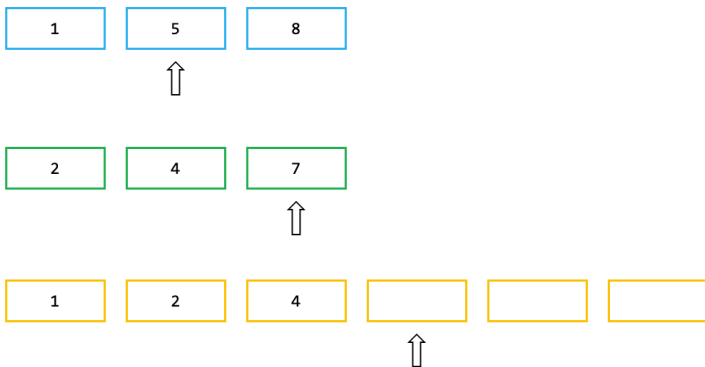
归并排序



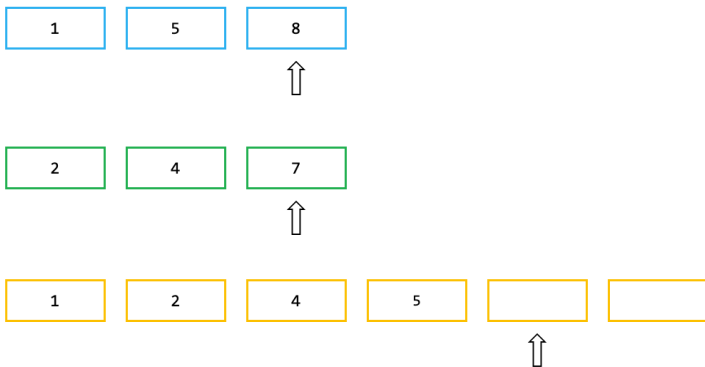
归并排序



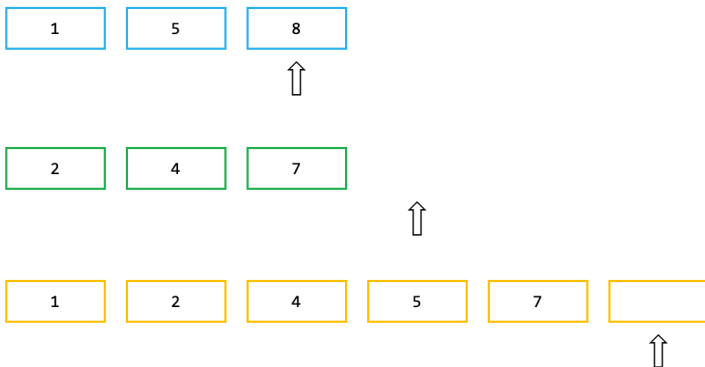
归并排序



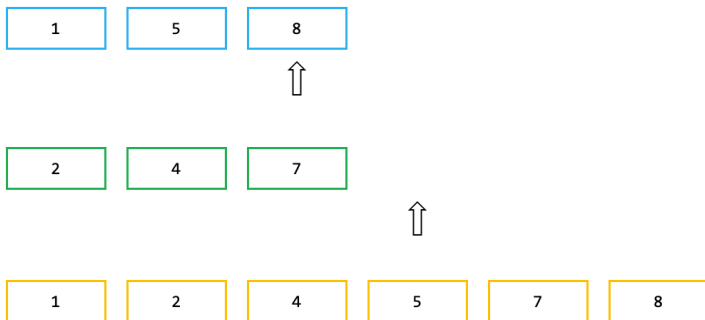
归并排序



归并排序



归并排序



归并排序

```
1  const int N = 1e5 + 5;
2  int n, a[N], b[N];
3
4  void merge(int l, int mid, int r) {
5      int x = l, y = mid + 1, z = l;
6      while(x <= mid && y <= r) {
7          if(a[x] <= a[y]) b[z++] = a[x++];
8          else b[z++] = a[y++];
9      }
10     while(x <= mid) b[z++] = a[x++];
11     while(y <= r) b[z++] = a[y++];
12     for(int i = l; i <= r; ++i)
13         a[i] = b[i];
14 }
15
16 void merge_sort(int l, int r) {
17     if(l == r)
18         return;
19     int mid = (l + r) / 2;
20     merge_sort(l, mid); merge_sort(mid + 1, r);
21     merge(l, mid, r);
22 }
```

例题 3：逆序对

对于给定的长度为 n 的数组 a ，定义逆序对 (i, j) 满足： $i < j$ 且 $a_i > a_j$ 。
请求出 a 数组的逆序对总数。

对于所有数据， $1 \leq n \leq 5 \times 10^5$ ， $1 \leq a_i \leq 10^9$ 。

输入 #1

6
5 4 2 6 3 1

输出 #1

11

例题 3：逆序对

利用归并排序可以在 $\mathcal{O}(n \log n)$ 的时间内找出所有逆序对。

例题 3：逆序对

利用归并排序可以在 $\mathcal{O}(n \log n)$ 的时间内找出所有逆序对。

考虑合并前半部分与后半部分数组时，统计产生的逆序对数。

对于前半部分中的每一个元素，其产生的逆序对数为**后半部分中已被合并的元素个数**，对其累加即可。

每一个逆序对只会在其被合并时统计一次，因此是不重不漏的。

例题 3：逆序对

```
1  typedef long long ll;
2  const int N = 5e5 + 5;
3  int n, a[N], b[N];
4  ll ans;
5
6  void merge(int l, int mid, int r) {
7      int x = l, y = mid + 1, z = l;
8      while(x <= mid && y <= r) {
9          if(a[x] <= a[y]) b[z++] = a[x++], ans += (y - mid - 1);
10         else b[z++] = a[y++];
11     }
12     while(x <= mid) b[z++] = a[x++], ans += (r - mid);
13     while(y <= r) b[z++] = a[y++];
14     for(int i = l; i <= r; ++i)
15         a[i] = b[i];
16 }
17
18 void merge_sort(int l, int r) {
19     if(l == r)
20         return;
21     int mid = (l + r) / 2;
22     merge_sort(l, mid); merge_sort(mid + 1, r);
23     merge(l, mid, r);
24 }
```

目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

快速幂

计算 a^b ，最暴力的方法是乘 b 次，时间复杂度 $\mathcal{O}(b)$ 。

当 b 很大时（如 $b = 10^9$ ），暴力显然不可行。

核心思想

快速幂 (Binary Exponentiation) 利用分治思想，将指数不断折半。

- 若 b 为偶数: $a^b = (a^{b/2})^2$
- 若 b 为奇数: $a^b = a \cdot (a^{b/2})^2$

时间复杂度从 $\mathcal{O}(b)$ 降至 $\mathcal{O}(\log b)$ 。

快速幂

计算 2^6 。

1 6 是偶数: $2^6 = (2^3)^2$;

2 3 是奇数: $2^3 = 2 \times (2^1)^2$;

3 1 是奇数: $2^1 = 2 \times (2^0)^2$;

快速幂

计算 2^6 。

1 6 是偶数: $2^6 = (2^3)^2$;

2 3 是奇数: $2^3 = 2 \times (2^1)^2$;

3 1 是奇数: $2^1 = 2 \times (2^0)^2$;

另一种方式: 将指数 b 按二进制分解, 然后用 $a^1, a^2, a^4, \dots$ 去凑出 a^b 。

快速幂代码

实际应用中经常要求模 p ，取模不影响乘法运算。

```
1 // 快速幂: 计算  $a^b \% p$ 
2 // base: 用于存储  $a^1, a^2, a^4, \dots$ 
3 // res: 存储最终所求  $a^b$ 
4 long long quick_pow(long long a, long long b, long long p) {
5     long long base = a, res = 1 % p;
6     while(b > 0) {
7         if(b % 2 == 1) res = res * base % p;
8         base = base * base % p;
9         b /= 2;
10    }
11    return res;
12 }
```

矩阵快速幂

矩阵快速幂 = 快速幂 + 矩阵乘法。

为什么可以对矩阵使用快速幂？

因为**矩阵乘法满足结合律**：

$$(A \times B) \times C = A \times (B \times C)$$

所以计算 A^k 也可以像普通快速幂一样折半计算！

典型应用：**加速线性递推**，如斐波那契数列。

矩阵快速幂应用：斐波那契

斐波那契数列： $F_1 = 1, F_2 = 1, F_n = F_{n-1} + F_{n-2}$

可以写成矩阵形式：

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix}$$

因此：

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-2} \times \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

用矩阵快速幂计算 F_n ，时间复杂度 $\mathcal{O}(\log n)$ 。

目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

什么是 RMQ?

RMQ (Range Maximum/Minimum Query): 区间最值查询问题。

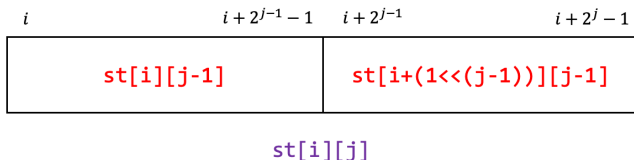
ST 表

ST 表 (Sparse Table) 是基于**倍增**思想解决 RMQ 问题的数据结构。

- 预处理: $\mathcal{O}(n \log n)$
- 每次查询: $\mathcal{O}(1)$

ST 表预处理

定义 $st_{i,j}$: 从 i 开始, 长度为 2^j 的区间中的最大值。



递推公式:

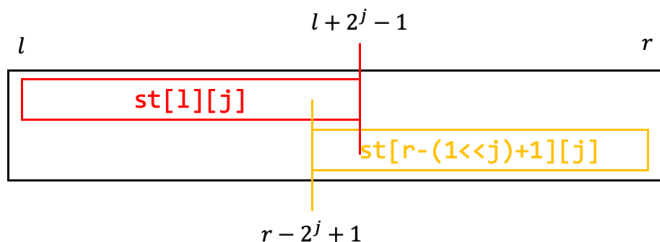
$$st_{i,j} = \max(st_{i,j-1}, st_{i+2^{j-1},j-1})$$

即: 把长度为 2^j 的区间, 分成两个长度为 2^{j-1} 的区间分别求最值。

ST 表查询

查询区间 $[l, r]$ 的最大值：

设 $j = \lfloor \log_2(r - l + 1) \rfloor$ ，即最大的满足 $2^j \leq$ 区间长度的 j 。



答案 = $\max(st_{l,j}, st_{r-2^j+1,j})$

两个长度为 2^j 的区间 **覆盖** 了整个查询区间（允许重叠，最值不受影响）。

ST 表

```
1  const int N = 1e5 + 5, LOG2 = 20;
2  int n, m, a[N], st[N][LOG2];
3
4  void preprocess() {
5      for(int i = 1; i <= n; ++i)
6          st[i][0] = a[i];
7      for(int j = 1; (1 << j) <= n; ++j)
8          for(int i = 1; i + (1 << j) - 1 <= n; ++i)
9              st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
10 }
11
12 int rmq(int l, int r) {
13     int j = __lg(r - l + 1);
14     return max(st[l][j], st[r - (1 << j) + 1][j]);
15 }
```

目录

1 什么是分治

2 二分查找

3 二分答案

4 归并排序

5 快速幂与矩阵快速幂

6 倍增与 RMQ

7 习题

习题

- P1115 最大子段和
- P1873 [COCI 2011/2012 #5] EKO / 砍树
- P1939 矩阵加速（数列）
- P3865 【模板】ST 表 & RMQ 问题

鸣谢

- 感谢西安交通大学算法竞赛校队以及“这是一个ACM队”中的同学支持。
- 参考文献：往年课件、OI-wiki 等。

Thanks for listening.